

```

#ifndef METRO_H
#define METRO_H

#include <string>
#include <map>
#include <vector>

using namespace std;

// Structure to store route details
struct Edge {
    int to;
    int time;
    int fare;
};

class DelhiMetro {
    int V; // Number of stations
    map<int, string> stationNames;
    vector<vector<Edge>> adj;

public:
    DelhiMetro(int v);
    void addStation(int id, string name);
    void addRoute(int u, int v, int time, int fare);
    void displayStations();
    void findShortestRoute(int start, int end);
};

#endif

#include "metro.h"
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>

```

```

using namespace std;

// Constructor: Initializing Adjacency List
DelhiMetro::DelhiMetro(int v) : V(v) {
    adj.resize(v);
}

void DelhiMetro::addStation(int id, string name) {
    stationNames[id] = name;
}

// Bidirectional routing
void DelhiMetro::addRoute(int u, int v, int time, int fare) {
    if (u < V && v < V) {
        adj[u].push_back({v, time, fare});
        adj[v].push_back({u, time, fare});
    }
}

void DelhiMetro::displayStations() {
    cout << "\n--- AVAILABLE METRO STATIONS ---" << endl;
    for (auto const& [id, name] : stationNames) {
        cout << "[" << id << "]" << name << endl;
    }
}

void DelhiMetro::findShortestRoute(int start, int end) {
    // Validation
    if (stationNames.find(start) == stationNames.end() || stationNames.find(end) ==
stationNames.end()) {
        cout << "\n[Error] Invalid Station ID entered!" << endl;
        return;
    }

    // Dijkstra's Requirements
    vector<int> dist(V, 2147483647); // Initializing with INT_MAX

```

```

vector<int> parent(V, -1);
vector<int> totalFare(V, 0);

// Min-Priority Queue (Priority based on Time)
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;

dist[start] = 0;
pq.push({0, start});

while (!pq.empty()) {
    int d = pq.top().first;
    int u = pq.top().second;
    pq.pop();

    // Optimization: Stop if destination is reached
    if (u == end) break;

    // Skip if a better path is already processed
    if (d > dist[u]) continue;

    for (auto& edge : adj[u]) {
        // Relaxation Step
        if (dist[edge.to] > dist[u] + edge.time) {
            dist[edge.to] = dist[u] + edge.time;
            totalFare[edge.to] = totalFare[u] + edge.fare;
            parent[edge.to] = u;
            pq.push({dist[edge.to], edge.to});
        }
    }
}

// Output Result
if (dist[end] == 2147483647) {
    cout << "\nRoute not found!" << endl;
} else {
    cout << "\n===== " << endl;
}

```

```

cout << "    JOURNEY SUMMARY    " << endl;
cout << "===== " << endl;
cout << "TIME: " << dist[end] << " mins" << endl;
cout << "FARE: Rs. " << totalFare[end] << endl;

// Path Reconstruction
vector<int> path;
for (int v = end; v != -1; v = parent[v]) path.push_back(v);
reverse(path.begin(), path.end());

cout << "\nOPTIMAL ROUTE:" << endl;
for (int i = 0; i < path.size(); i++) {
    cout << stationNames[path[i]] << (i == path.size() - 1 ? "" : " -> ");
}
cout << "\n\nTotal Stations: " << path.size();
cout << "\n===== \n";
}
}

#include "metro.h"
#include <iostream>

using namespace std;

int main() {
    // Creating Metro System with capacity for 100 stations
    DelhiMetro dm(100);

    // Adding Stations
    dm.addStation(0, "Dwarka Sec-21");
    dm.addStation(7, "Rajiv Chowk");
    dm.addStation(15, "Mandi House");
    dm.addStation(10, "Yamuna Bank");
    dm.addStation(11, "Laxmi Nagar");
    dm.addStation(13, "Karkarduma");
    dm.addStation(14, "Anand Vihar");

```

```

dm.addStation(21, "Noida Electronic City");
dm.addStation(24, "Kashmere Gate");
dm.addStation(26, "New Delhi");
dm.addStation(28, "Central Secretariat");
dm.addStation(29, "INA");
dm.addStation(30, "Hauz Khas");

// Creating a Connected Network (The Spider Web)
dm.addRoute(0, 7, 30, 50); // Dwarka - Rajiv Chowk
dm.addRoute(7, 15, 5, 10); // Rajiv Chowk - Mandi House
dm.addRoute(15, 10, 5, 10); // Mandi House - Yamuna Bank
dm.addRoute(10, 11, 4, 10); // Yamuna Bank - Laxmi Nagar
dm.addRoute(11, 13, 3, 10); // Laxmi Nagar - Karkarduma
dm.addRoute(13, 14, 5, 10); // Karkarduma - Anand Vihar
dm.addRoute(10, 21, 20, 40); // Yamuna Bank - Noida
dm.addRoute(24, 26, 8, 20); // Kashmere Gate - New Delhi
dm.addRoute(26, 7, 3, 10); // New Delhi - Rajiv Chowk
dm.addRoute(15, 28, 8, 15); // Mandi House - Central Secretariat
dm.addRoute(28, 29, 6, 12); // Central Secretariat - INA
dm.addRoute(7, 30, 25, 45); // Rajiv Chowk - Hauz Khas
dm.addRoute(30, 21, 15, 30); // Hauz Khas - Noida (Shortcut)

// User Interface
dm.displayStations();

int src, dest;
cout << "\n[!] Enter Source Station ID: ";
cin >> src;
cout << "[!] Enter Destination Station ID: ";
cin >> dest;

dm.findShortestRoute(src, dest);

return 0;
}

```

